Tech Challenge 3

Build, Scale & Monitor a Containerized Application on Amazon EKS

Objective

To build a kubernetes cluster using EKS

To deploy your own containerized application

To expose it to the internet

To implement autoscaling

To monitor performance using cloudwatch

To understand how systems behave under load

Phase 1: Environment Setup

Step 1: Install Tools

-aws cli - The AWS CLI allows your terminal to communicate with AWS. During this project, we'll use it to interact with services such as **EKS, ECR, IAM, and CloudWatch**

-eksctl - eksctl is a command-line tool specifically designed to simplify creating and managing **Amazon EKS clusters**

```
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % echo "===== AWS CLI
====="
aws --version

echo "===== AWS AUTHENTICATION ====="
aws sts get-caller-identity

echo "===== AWS REGION ====="
aws configure get region

echo "===== EKSCTL ====="
eksctl version

echo "===== KUBECTL ====="
kubectl version --client

echo "===== AWS IAM AUTHENTICATOR ====="
aws-iam-authenticator version
===== AWS CLI =====
aws-cli/2.36.7 Python/3.14.7 Darwin/23.6.0 source/arm64
===== AWS AUTHENTICATION =====
{
    "UserId": "AIDA6GIEP7HTOV3H2UUEW",
    "Account": "975503882726",
    "Arn": "arn:aws:iam::975503882726:user/Learning"
}
===== AWS REGION =====
us-west-2
===== EKSCTL =====
0.230.0
===== KUBECTL =====
Client Version: v1.37.0
Kustomize Version: v5.8.1
===== AWS IAM AUTHENTICATOR =====
{"Version":"0.7.20","Commit":"Homebrew"}
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS %
```

Achievement

-aws --version
-Your AWS credentials work and AWS authentication succeeds
-Your project is configured for `us-west-2`
`-eksctl is installed`
`-Kubernetes CLI is installed`
`-AWS/Kubernetes authentication helper is installed`

Step 2: Configure AWS

aws configure - It shows where AWS CLI is getting its configuration and credentials from without displaying your full secret key

```
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % aws configure list
NAME       : VALUE                    : TYPE             : LOCATION
profile    : <not set>                : None             : None
access_key : ****************EK7M     : shared-credentials-file :
secret_key : ****************RAWG     : shared-credentials-file :
region     : us-west-2                : config-file      : ~/.aws/config
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS %
```

Phase 2: Infrastructure (EKS)

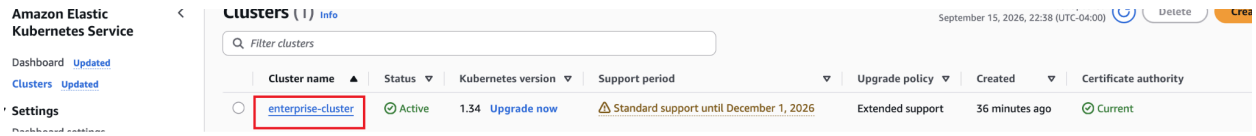
Step 3: Create Cluster

```
eksctl create cluster \
  --name enterprise-cluster \
  --region us-west-2 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 3 \
  --nodes-min 3 \
  --nodes-max 6 \
  --managed
```

```
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % aws eks describe-clus
ter \
  --name enterprise-cluster \
  --region us-west-2 \
  --query "cluster.status" \
  --output text
ACTIVE
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS %
```

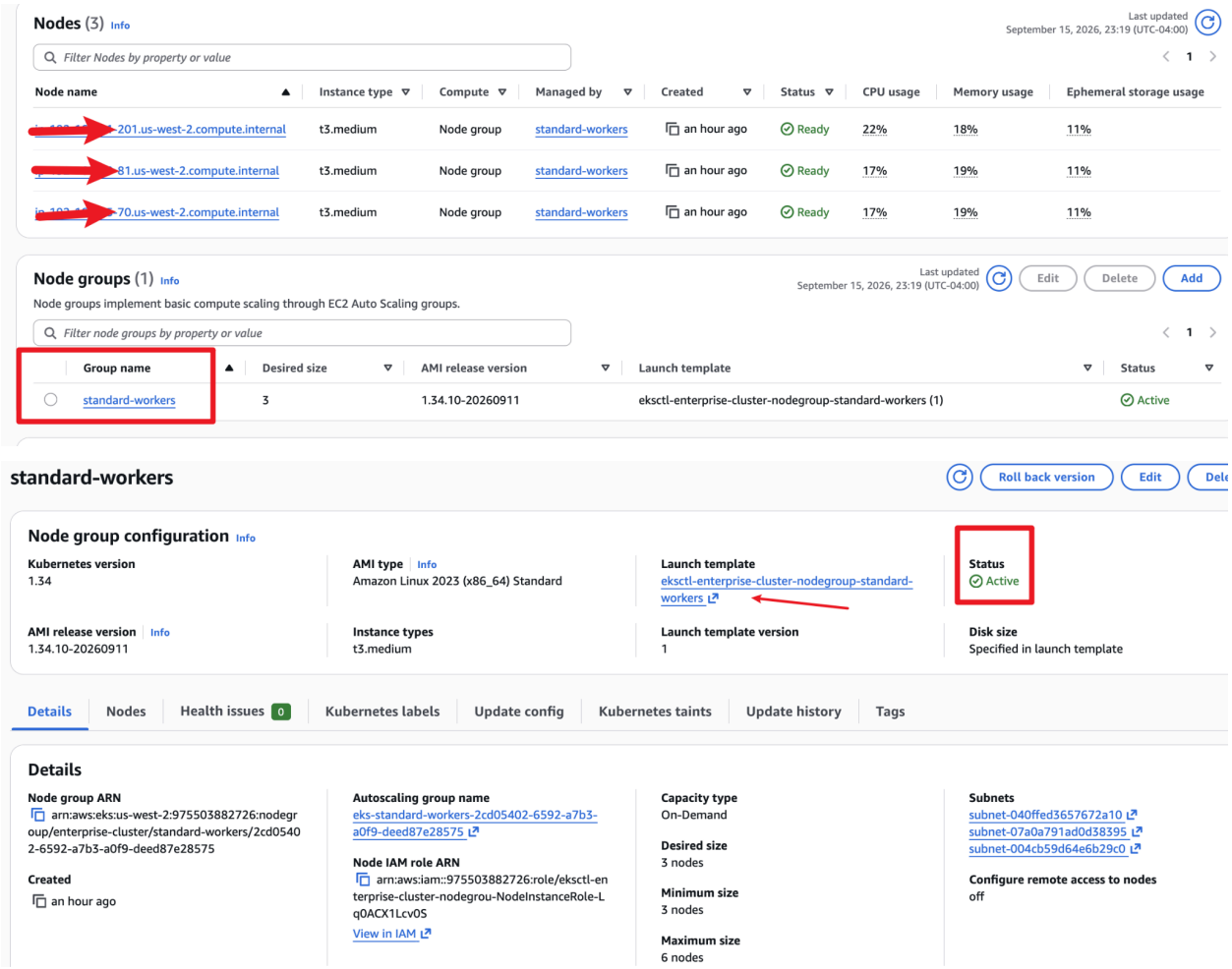
`- eksctl create cluster` tells `eksctl` to create a new Amazon EKS cluster. AWS will create the Kubernetes control plane and the supporting resources required for the cluster.

`--name enterprise-cluster` gives the EKS cluster the name " enterprise-cluster "

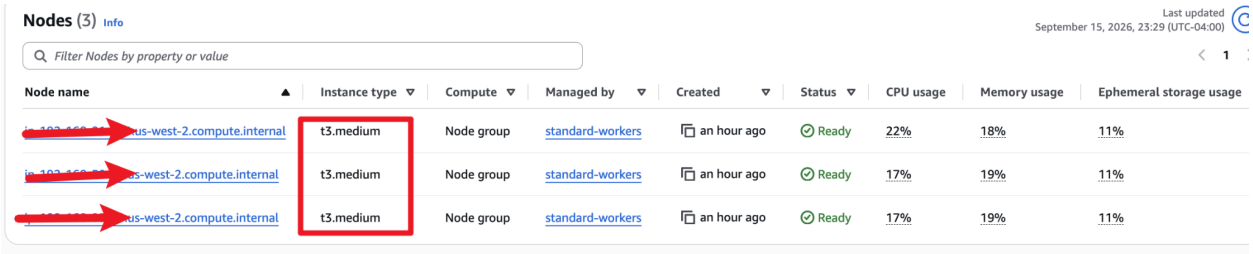


`--region us-east-1` tells AWS which region to create the cluster in. My region is us-west-2

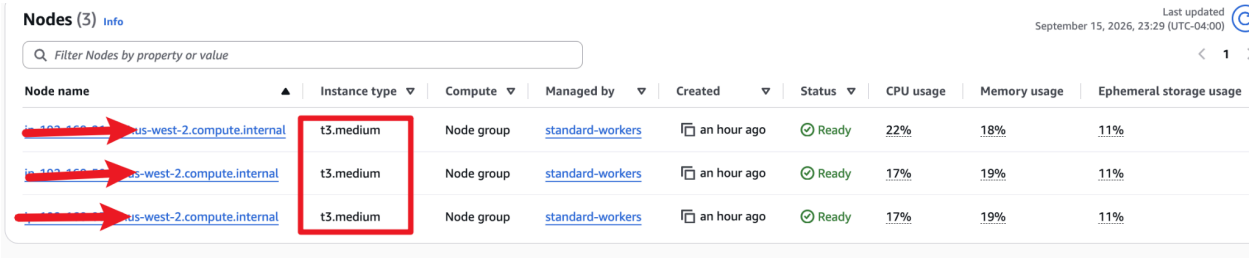
`--nodegroup-name standard-workers` creates a worker-node group named " standard-workers"



`--node-type t3.medium` specifies the EC2 instance type for each worker node. So the nodes created for this node group will be `t3.medium` instances.



`--nodes 3` tells EKS to start the node group with 3 worker nodes “Initial capacity = 3 EC2 nodes”



`--nodes-min 3` establishes the minimum node-group size "`--nodes-min 3` establishes the minimum node-group size" The node group should therefore not scale below three instances

`--nodes-max 6` establishes the maximum node-group size "Maximum = 6 nodes" the configured node-group capacity range is 3 nodes to 6 nodes

`--managed` tells `eksctl` to create an EKS managed node group. AWS handles much of the lifecycle management of the EC2 worker nodes, rather than me manually managing the instances yourself.

Explain why:

t3.medium - `t3.medium` is the EC2 instance type used for each worker node in your cluster. It provides a reasonable amount of CPU and memory for a smaller EKS environment without choosing a much larger instance, it also give Kubernetes enough compute resources to run the application, Kubernetes system components, monitoring agents, and other workloads

3 nodes - Three nodes provide more capacity and better resilience than running everything on one machine. Kubernetes can distribute workloads across the nodes. If one worker node becomes unavailable, workloads can potentially be rescheduled onto the remaining nodes, assuming they have enough capacity.

when would I scale nodes - Scale up when your Pods need more CPU or memory than the existing nodes can provide, and scale down when demand decreases.

Step 4: Connect to Cluster -

```
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % aws eks update-kubeconfig \
  --region us-west-2 \
  --name enterprise-cluster
Added new context arn:aws:eks:us-west-2:975503882726:cluster/enterprise-cluster to /Users/greatness/
.kube/config
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % kubectl get nodes
NAME                                           STATUS   ROLES    AGE   VERSION
ip-192-168-21-201.us-west-2.compute.internal   Ready    <none>   81m   v1.34.10-eks-cb19647
ip-192-168-50-81.us-west-2.compute.internal    Ready    <none>   81m   v1.34.10-eks-cb19647
ip-192-168-85-70.us-west-2.compute.internal    Ready    <none>   81m   v1.34.10-eks-cb19647
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS %
```

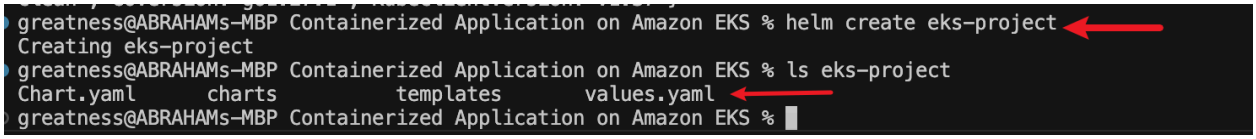
-The cluster connection allows **`kubectl`** on my Mac to communicate with my Amazon EKS cluster - My Mac → AWS CLI → EKS Cluster → Kubernetes Nodes

- The `update-kubeconfig` command saves the cluster connection information locally, and `kubectl get nodes` verifies that I can communicate with the cluster and see its worker nodes.

Step 6: Install Helm - Helm is a package manager for Kubernetes. It makes it easier to install, configure, upgrade, and manage applications running inside your EKS cluster using packages called Helm Charts

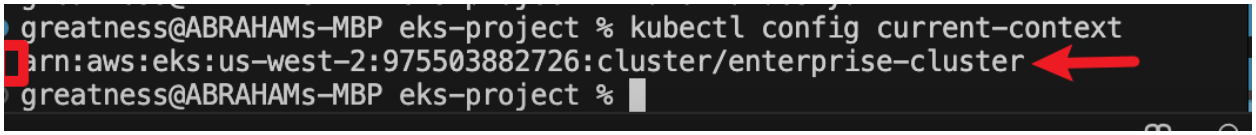


Step 7: Create Helm Chart - " **`helm create eks-project`** **"** creates the basic Helm Chart structure that we will customize to deploy and manage your containerized application on EKS.



I defined CPU and memory requests/limits to control resource usage, protect the worker nodes, and provide predictable resources for my application and autoscaling

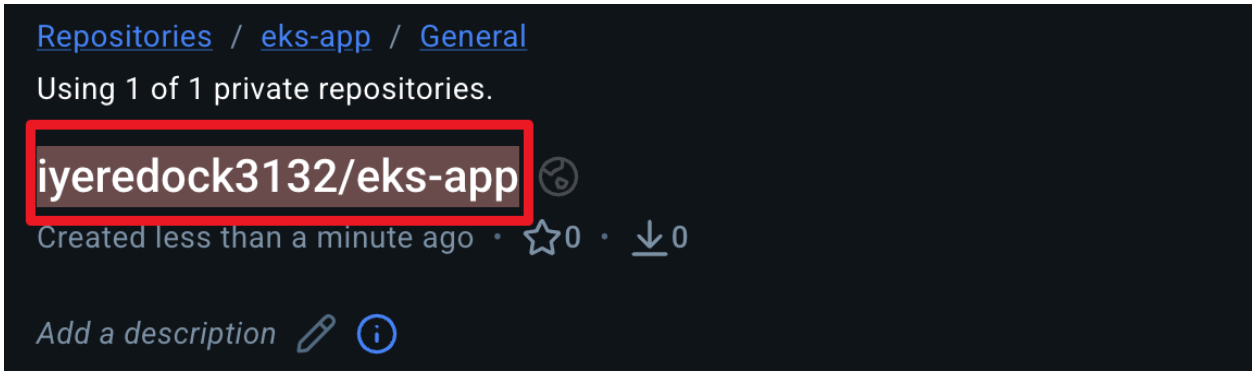
Step 8: Deploy Application- Firstly, I verified my Kubernetes context to confirm that `kubectl` was connected to the correct EKS cluster before deploying the application



Before deploying to EKS, I needed an **application for Kubernetes to run**. I created a simple Python Flask application along with a `requirements.txt` file for its dependencies and a `Dockerfile` containing the instructions needed to containerize the application.

```
greatness@ABRAHAMs-MBP app % ls -l
total 24
-rw-r--r--@ 1 greatness  staff  167 Sep 16 22:02 Dockerfile
-rw-r--r--@ 1 greatness  staff  279 Sep 16 22:01 app.py
-rw-r--r--@ 1 greatness  staff   13 Sep 16 22:02 requirements.txt
```

I also created a dedicated Docker Hub repository, **`iyeredock3132/eks-app`**. The repository is needed to store the Docker image after it is built, so that my EKS cluster can later pull the image from Docker Hub and deploy the application as Kubernetes Pods



Docker Image Build

I successfully built a **Docker image for my Flask application** using the Dockerfile I created earlier. The Dockerfile packaged the application, Python runtime, Flask dependency, and required files into a portable container image.

I built the image for the **`linux/amd64` architecture** so it is compatible with my EKS `t3.medium` worker nodes. The completed image was tagged as: iyeredock3132/eks-app:1.0

```
greatness@ABRAHAMs-MBP app % docker images | grep eks-app
iyeredock3132/eks-app:1.0                 383672c84167       51.2MB         51.2MB

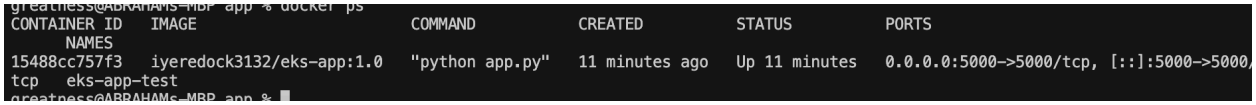
greatness@ABRAHAMs-MBP app %
```

I successfully **built and tested my Docker container locally** before deploying it to Amazon EKS. The Docker image was created as: iyeredock3132/eks-app:1.0

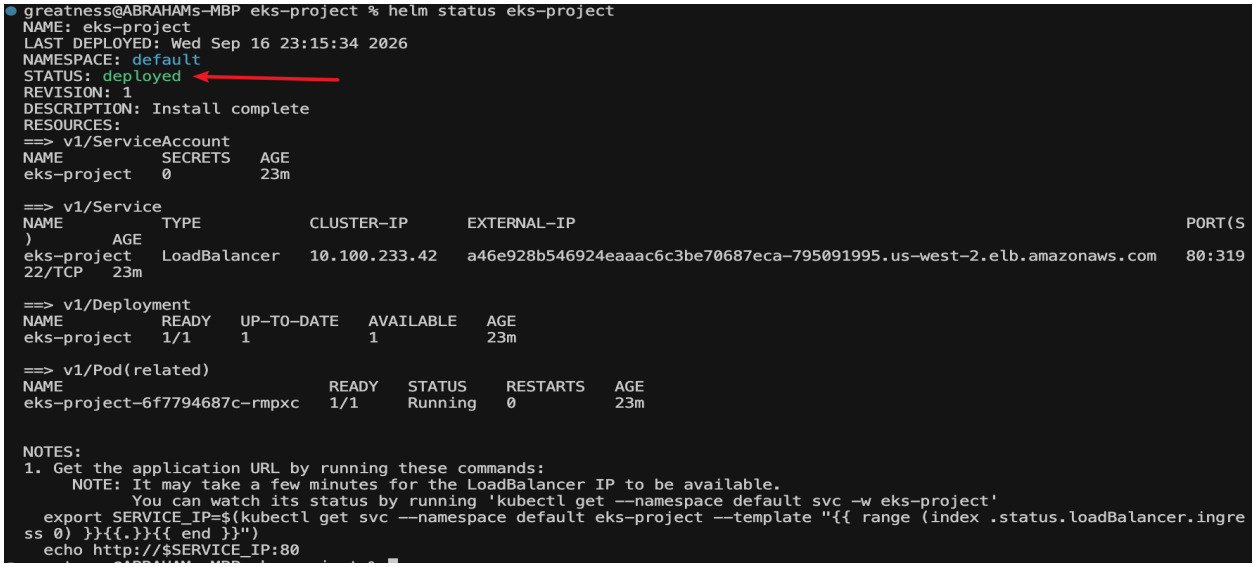
I then started a container named **`eks-app-test`** and mapped **port 5000 on my Mac to port 5000 inside the container**:



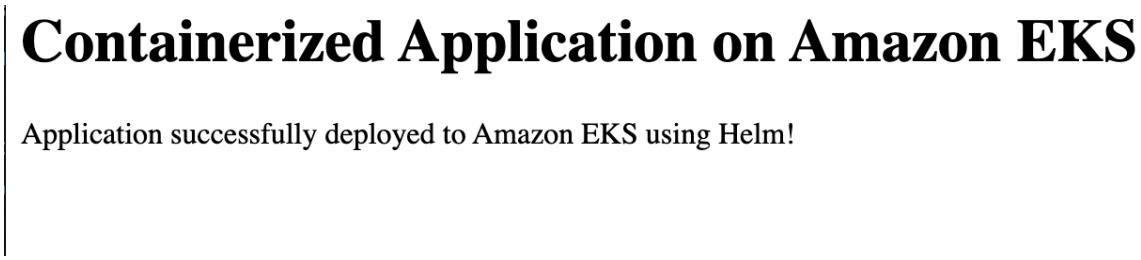
I confirmed that the container was **running successfully**. I also built the image for **`linux/amd64`** so it is compatible with the `t3.medium` worker nodes in my EKS cluster



I deployed my containerized application to Amazon EKS using Helm and exposed it publicly through a Kubernetes LoadBalancer Service. I verified the Helm release, running EKS Pod, LoadBalancer endpoint, and successful HTTP response from the application

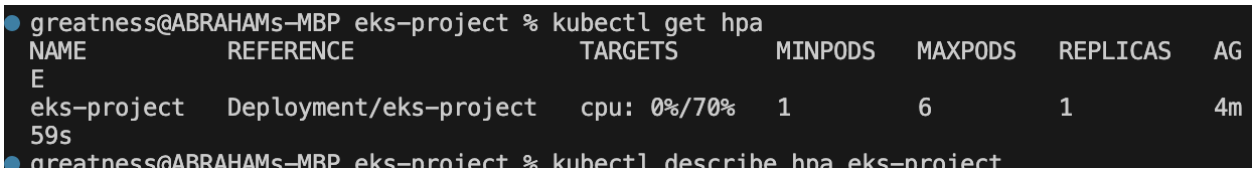


App LoadBalancer Service-

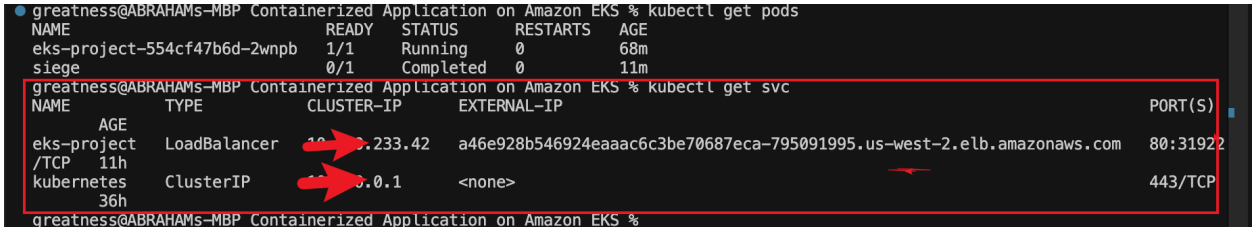


Phase 4: Autoscaling

Step 10: Create HPA - HPA (Horizontal Pod Autoscaler) automatically changes the number of Kubernetes Pods running your application based on demand. I configured HPA to automatically scale my application between 1 and 6 Pods based on CPU utilization, helping the application handle increased traffic while reducing unnecessary resources when demand is low.



Step 11: Load Testing - `kubectl get svc` displays the **Kubernetes Services** running in my current namespace. It is mainly to verify that my application is exposed through an AWS Load Balancer.

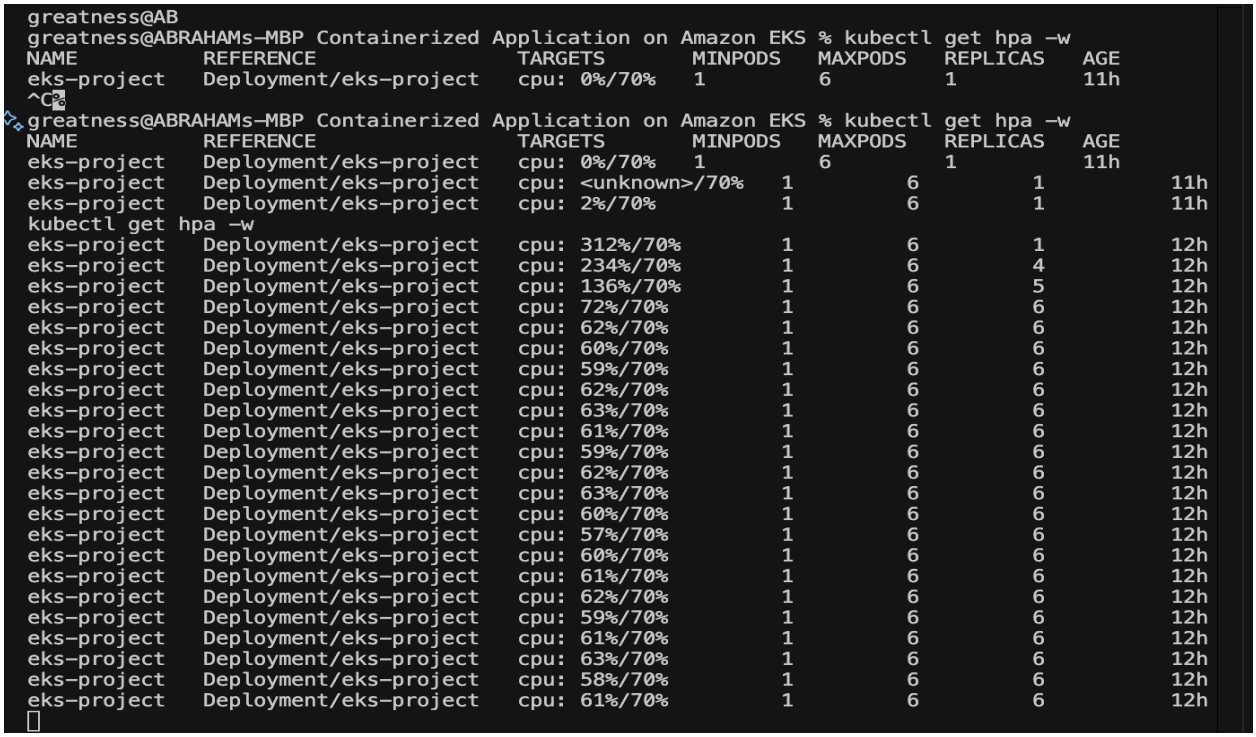


Load Testing and HPA Validation

I tested my Amazon EKS application to confirm that the **Horizontal Pod Autoscaler (HPA)** could automatically scale the application during increased traffic. My initial load test using **Siege** failed because it could not successfully connect to the application, resulting in **0 successful requests and 1,024 failed requests**.

I replaced Siege with a **BusyBox load-generator Pod**, which continuously sent HTTP requests to the application through the internal Kubernetes Service. I also adjusted the application's CPU request from **300m to 50m** to make the HPA test appropriate for the lightweight application.

During the successful load test, CPU utilization increased and the HPA automatically scaled the application from **1 Pod to the configured maximum of 6 Pods**. I verified the scaling using `kubectl get hpa -w`, `kubectl get pods -w`, and `kubectl top pods`.



```
greatness@AB
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % kubectl get hpa -w
NAME          REFERENCE                  TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
eks-project   Deployment/eks-project   cpu: 0%/70%   1         6         1          11h
^C
greatness@ABRAHAMs-MBP Containerized Application on Amazon EKS % kubectl get hpa -w
NAME          REFERENCE                  TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
eks-project   Deployment/eks-project   cpu: 0%/70%   1         6         1          11h
eks-project   Deployment/eks-project   cpu: <unknown>/70%   1         6         1          11h
eks-project   Deployment/eks-project   cpu: 2%/70%          1         6         1          11h
kubectl get hpa -w
eks-project   Deployment/eks-project   cpu: 312%/70%        1         6         1          12h
eks-project   Deployment/eks-project   cpu: 234%/70%        1         6         4          12h
eks-project   Deployment/eks-project   cpu: 136%/70%        1         6         5          12h
eks-project   Deployment/eks-project   cpu: 72%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 62%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 60%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 59%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 62%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 63%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 61%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 59%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 62%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 63%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 60%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 57%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 60%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 61%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 62%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 59%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 61%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 63%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 58%/70%         1         6         6          12h
eks-project   Deployment/eks-project   cpu: 61%/70%         1         6         6          12h
```

Workloads: Pods (17)

Last updated September 17, 2026, 12:40 (UTC-04:00) View details

Pod is the smallest and simplest Kubernetes object. A Pod represents a set of running containers on your cluster. Learn more

Filter namespaces | Filter Pods by name

▶ Advanced query filters

1 2

Name	Created	Status
aws-node-2bktd	September 15, 2026, 22:22 (UTC-04:00)	Running
aws-node-bvn5j	September 15, 2026, 22:22 (UTC-04:00)	Running
aws-node-h4s6d	September 15, 2026, 22:22 (UTC-04:00)	Running
coredns-7cd6945668-2rrtv	September 15, 2026, 22:19 (UTC-04:00)	Running
coredns-7cd6945668-5xgn9	September 15, 2026, 22:19 (UTC-04:00)	Running
eks-project-644c4cc6cf-2tt85	8 minutes ago	Running
eks-project-644c4cc6cf-84h5j	8 minutes ago	Running
eks-project-644c4cc6cf-bn8vf	8 minutes ago	Running
eks-project-644c4cc6cf-bnqrh	8 minutes ago	Running
eks-project-644c4cc6cf-hns7h	21 minutes ago	Running

This test successfully demonstrated **automatic scaling, load distribution, and Kubernetes resource monitoring on Amazon EKS**

```
NAME                              READY   STATUS    RESTARTS   AGE
eks-project-644c4cc6cf-2tt85      1/1     Running   0          18m
eks-project-644c4cc6cf-84h5j      1/1     Running   0          18m
eks-project-644c4cc6cf-bn8vf      1/1     Running   0          18m
eks-project-644c4cc6cf-bnqrh      1/1     Running   0          18m
eks-project-644c4cc6cf-hns7h      1/1     Running   0          31m
eks-project-644c4cc6cf-rddbp      1/1     Running   0          18m
load-generator                    1/1     Running   0          19m
```

```
greatness@ABRAHAMs-MBP eks-project % kubectl top pods
NAME                            CPU(cores)   MEMORY(bytes)
eks-project-644c4cc6cf-2tt85    30m          21Mi
eks-project-644c4cc6cf-84h5j    36m          21Mi
eks-project-644c4cc6cf-bn8vf    31m          21Mi
eks-project-644c4cc6cf-bnqrh    34m          21Mi
eks-project-644c4cc6cf-hns7h    32m          21Mi
eks-project-644c4cc6cf-rddbp    26m          21Mi
load-generator                  262m         2Mi
greatness@ABRAHAMs-MBP eks-project %
```

Phase 5: Break It

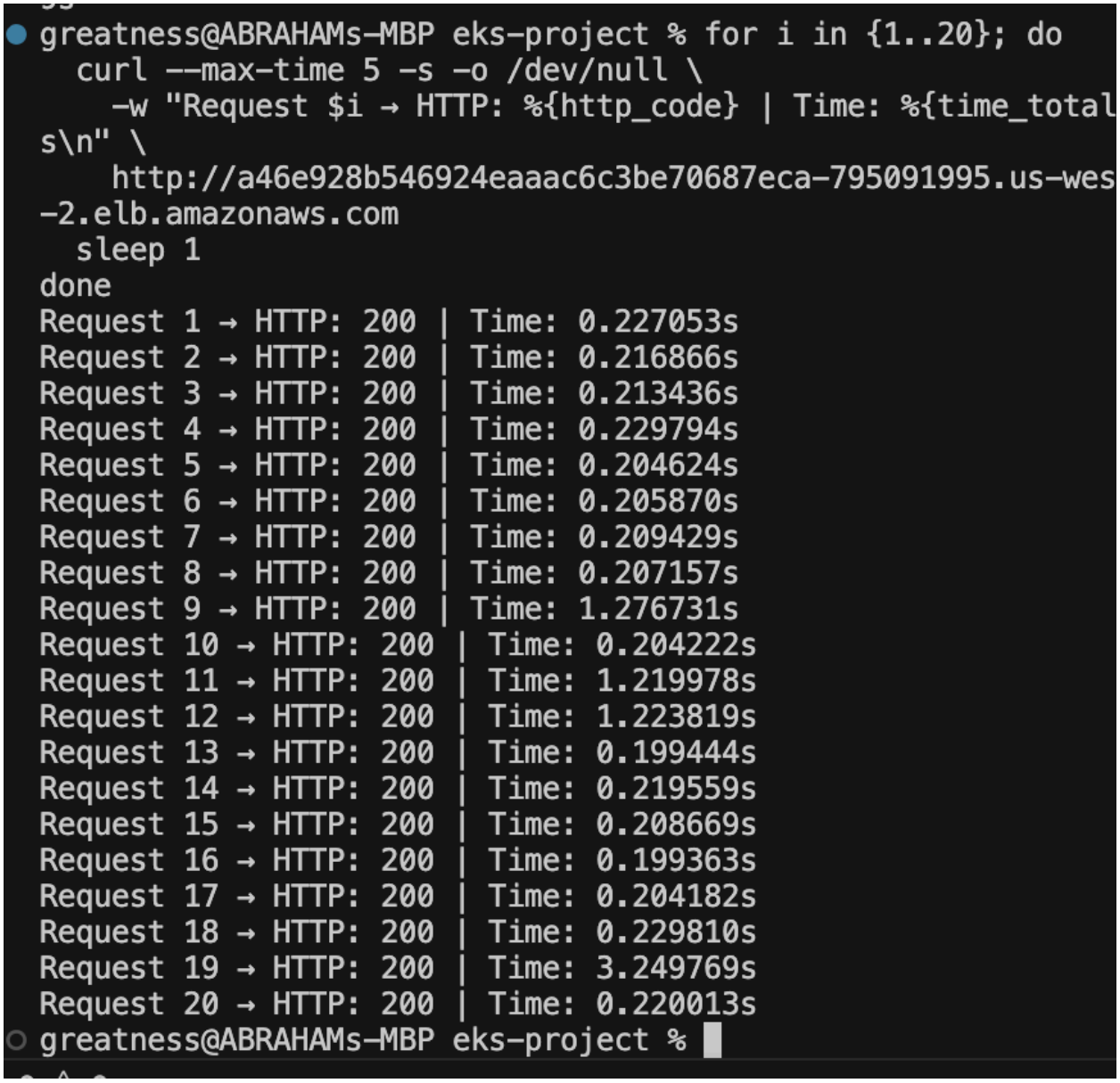
disable HPA

HPA Disabled Verification - I disabled the Horizontal Pod Autoscaler (HPA) and verified it using `kubectl get hpa`, which returned no HPA resources. I also confirmed that the application Deployment was running with only 1 replica. This created a controlled environment to test how the application behaves under load without automatic scaling.

```
get hpa
No resources found in default namespace.
get deployment eks-project
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eks-project   1/1     1            1           21h
greatness@ABRAHAMs-MBP eks-project %
```

Break-It Test: I disabled the Horizontal Pod Autoscaler and increased traffic using multiple load-generator Pods. Because HPA was disabled, the application remained at one replica while CPU usage increased to approximately **463m out of its 500m CPU limit**. The application remained available and continued returning HTTP 200 responses, but some response times increased from approximately **0.20–0.27 seconds to over 1.2 seconds**. This demonstrated performance degradation under CPU pressure. The test showed why autoscaling is important: without HPA, additional Pods were not created to distribute increased workload

```
NAME                                CPU(cores)
MEMORY(bytes)
eks-project-644c4cc6cf-2tt85   460m
22Mi
load-generator-1                    22m
0Mi
load-generator-10                   21m
0Mi
load-generator-2                    23m
0Mi
load-generator-3                    216m
2Mi
load-generator-4                    28m
0Mi
load-generator-5                    14m
0Mi
load-generator-6                    217m
2Mi
load-generator-7                    18m
0Mi
load-generator-8                    23m
0Mi
load-generator-9                    227m
2Mi
```

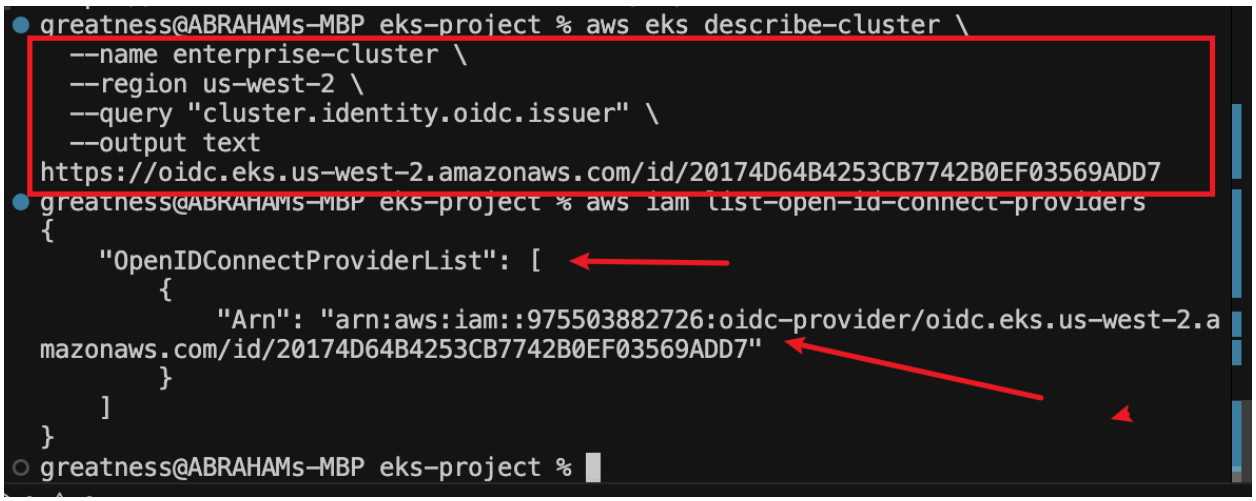


Phase 6: Monitoring (CloudWatch)

Step 12: Enable OIDC - Enabling the IAM OIDC provider is commonly needed when Kubernetes service accounts need to assume AWS IAM roles, including some EKS monitoring setups. It associates my `enterprise-cluster` with an **IAM OpenID Connect (OIDC) provider** in AWS

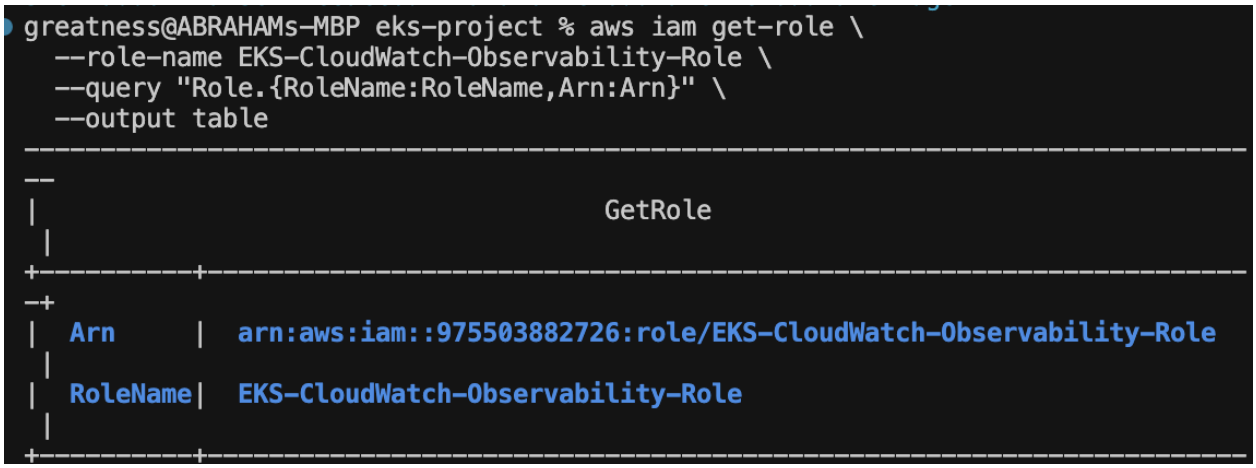
OIDC Verification: I verified that the IAM OIDC provider was successfully associated with my EKS cluster by comparing the EKS cluster's OIDC issuer ID with the OIDC provider registered in

AWS IAM. The matching provider confirms that the cluster can use OIDC-based IAM authentication for appropriately configured Kubernetes workloads



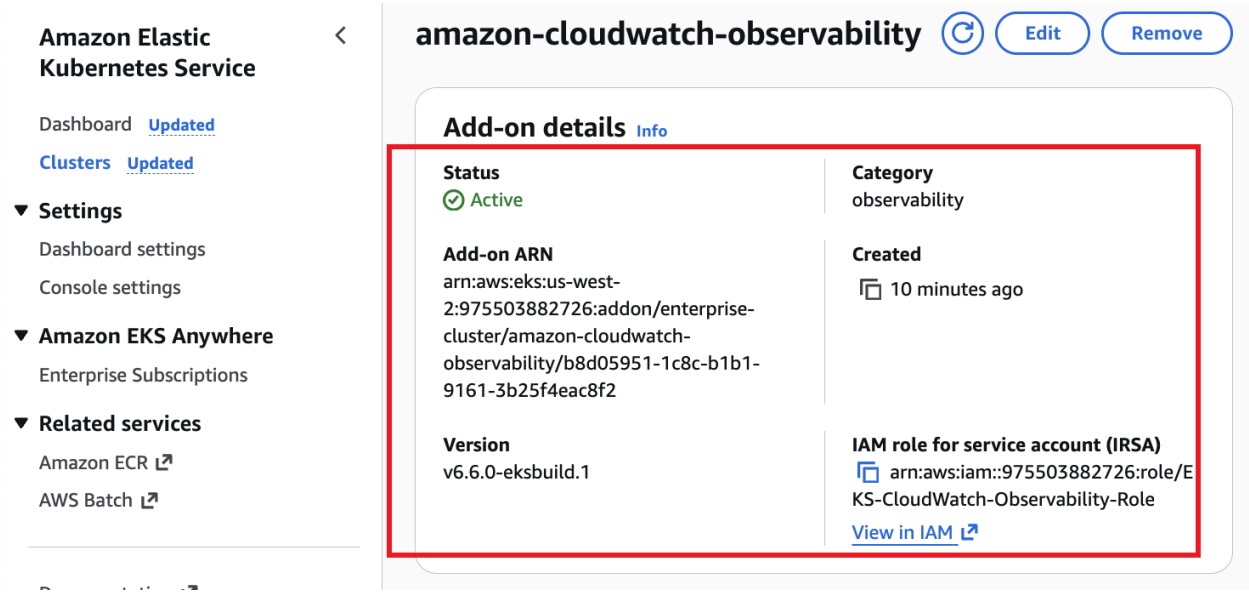
Step 13: Create IAM Role&Policy

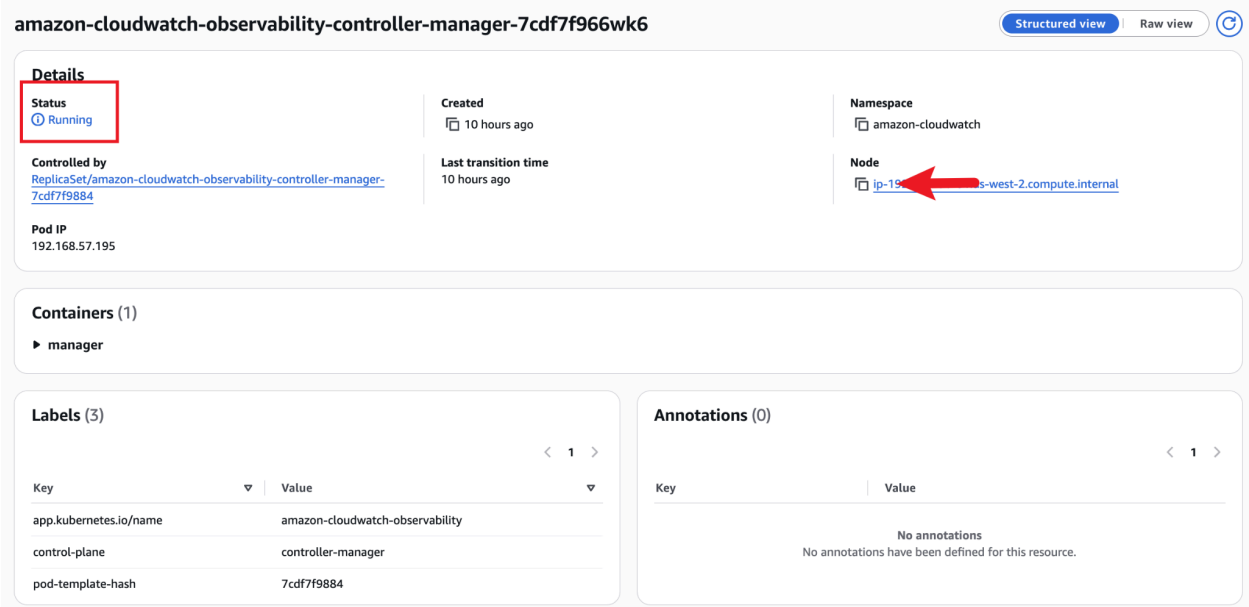
I created an IAM role for the CloudWatch Agent using `eksctl` and IRSA. The role is associated with the `cloudwatch-agent` Kubernetes identity and has the AWS-managed `CloudWatchAgentServerPolicy`. This allows the monitoring workload to securely send EKS metrics and monitoring data to Amazon CloudWatch without storing AWS credentials inside the Pods.



```
greatness@ABRAHAMs-MBP eks-project % aws iam list-attached-role-policies \
  --role-name EKS-CloudWatch-Observability-Role \
  --query "AttachedPolicies[*].{PolicyName:PolicyName,PolicyArn:PolicyArn}" \
  --output table
-------------------------------------------------------------------------
|                        ListAttachedRolePolicies                       |
+------------+----------------------------------------------------------+
|  PolicyArn |  arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy     |
|  PolicyName|  CloudWatchAgentServerPolicy                             |
+------------+----------------------------------------------------------+
```

Step 15: Deploy CloudWatch Agent - Deploy CloudWatch Agent: I deployed the Amazon CloudWatch Observability add-on to my EKS cluster. The add-on deployed CloudWatch Agent Pods across the worker nodes to collect Kubernetes infrastructure metrics. I verified the deployment using `kubectl get pods` and `kubectl get daemonset` and confirmed that the CloudWatch Agent Pods were running successfully.





Step 16: Verify Monitoring

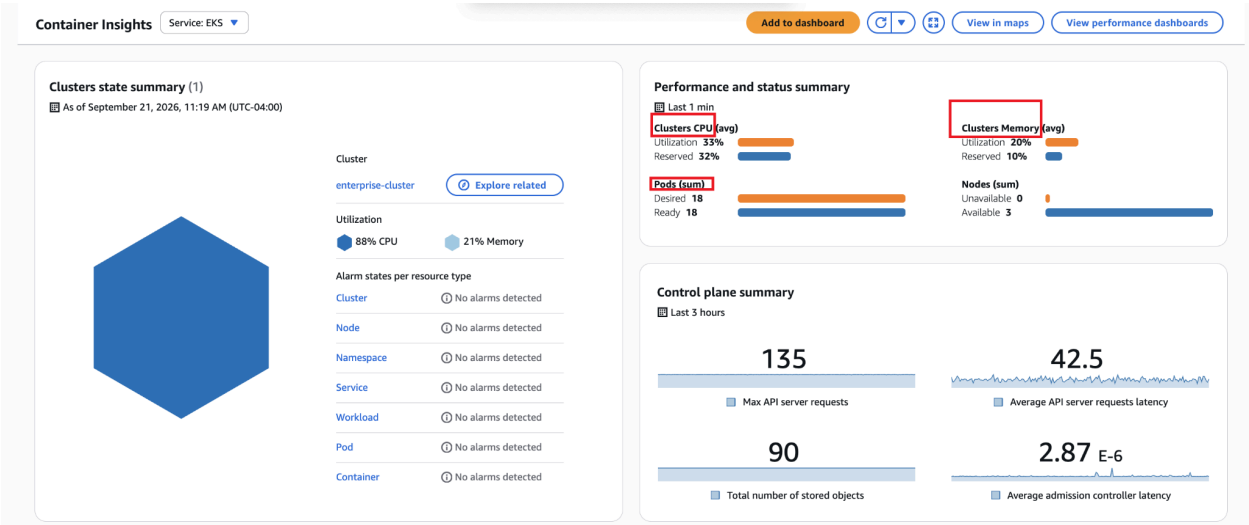
```
greatness@ABRAHAMs-MBP eks-project % kubectl get pods -n amazon-cloudwatch
NAME                                                              READY   STATUS    RESTARTS
 AGE
amazon-cloudwatch-observability-controller-manager-7cdf7f966wk6   1/1     Running   0
 10h
cloudwatch-agent-6wjdj                                            1/1     Running   0
 10h
cloudwatch-agent-h54bd                                            1/1     Running   0
 10h
cloudwatch-agent-snfxf                                            1/1     Running   0
 10h
fluent-bit-ssgxk                                                  1/1     Running   0
 10h
fluent-bit-wfq4d                                                  1/1     Running   0
 10h
fluent-bit-wn4j8                                                  1/1     Running   0
 10h
greatness@ABRAHAMs-MBP eks-project %
```

Observe-

cpu usage - The cluster summary shows 88% CPU utilization, while the performance panel shows a 33% average CPU utilization for its selected “Last 1 min” view. These numbers can differ because the panels represent different calculations/views. The important observation is that CPU activity is being captured successfully by CloudWatch. Higher CPU utilization means the cluster's processors are doing more work, such as processing application traffic, Kubernetes services, and monitoring workloads.

-memory usage - The cluster summary shows approximately **21% memory utilization**, while the performance panel shows **20% average memory utilization**. This indicates that only about one-fifth of the measured memory capacity is currently being used.

-pod behavior - CloudWatch reports **18 Desired Pods and 18 Ready Pods**. This is a healthy result: Kubernetes currently has all the Pods it expects, and all of them are ready to serve or perform their assigned work.

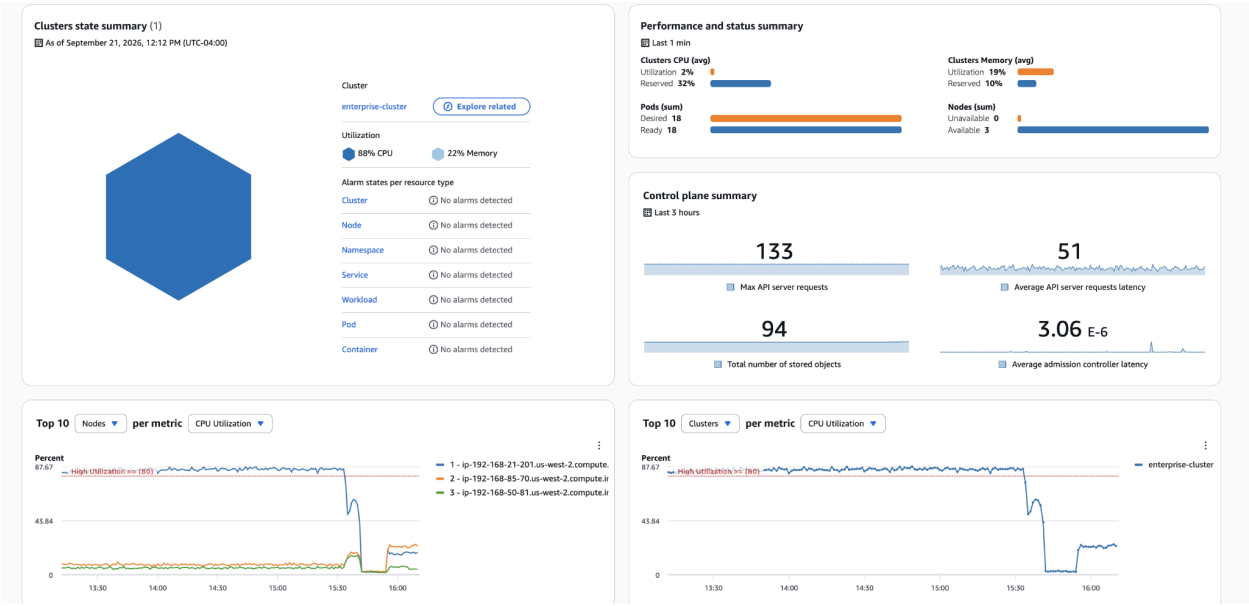


Step 18: Monitor Under Load

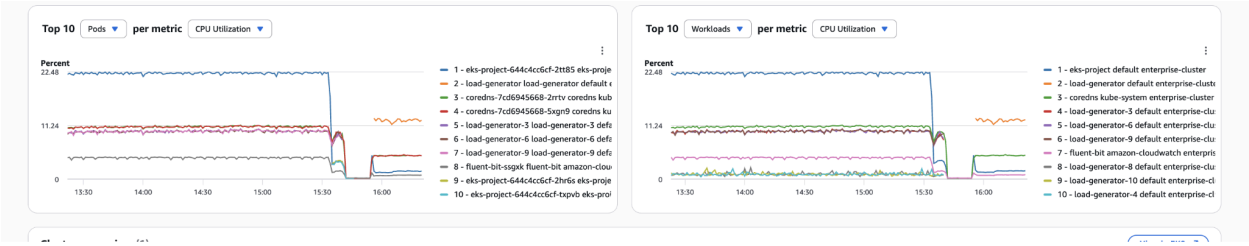
cpu spike

pod scaling

system response



pod scaling



The scaling was triggered by **CPU utilization**. My HPA is configured withTarget CPU = 70% Minimum Pods = 1 Maximum Pods = 6 . Before the load test, I had CPU: 2%/70% Replicas: 1

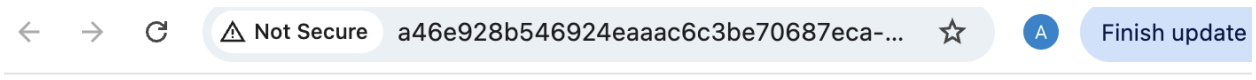
During the load test, CPU increased toward/above the **70% target**, causing HPA to increase the replicas. You later observed: CPU: 70%/70% Replicas: 5

Was there a delay?

Yes. CPU utilization was the metric that triggered autoscaling, with the HPA configured for a 70% CPU target. When load increased, CPU utilization rose and the HPA increased the application replicas from 1 to 5. There was a short delay because Kubernetes first had to detect

the increased CPU usage, calculate the required replicas, create the new Pods, and wait for them to become ready. After scaling, CPU stabilized around the 70% target

Access Application



Containerized Application on Amazon EKS

Application successfully deployed to Amazon EKS using Helm!

written answers

1 How Traffic Reaches the App

Users access the application through the AWS Load Balancer. The Load Balancer sends traffic to the Kubernetes Service, which then routes requests to the available application Pods running on EKS.

2. What Happens During Scaling

The HPA monitors CPU utilization with a target of 70%. When CPU demand increases, Kubernetes creates additional Pods to distribute the workload. In my test, the application scaled from 1 Pod to multiple Pods, reaching as many as 6, and scaled back down when demand decreased.

3. Role of Helm

Helm manages the Kubernetes deployment using reusable templates and configuration in `values.yaml`. I used Helm to deploy and update the application, Service, resource limits, and HPA settings without manually managing each Kubernetes resource.

4. What CloudWatch Showed

CloudWatch Container Insights showed the cluster's CPU and memory utilization, Pod count, and node health. During load testing, I could see increased CPU activity and changes in the number of Pods as the HPA responded to the workload.

5. What I Would Improve

I would improve the project by adding CloudWatch alarms, HTTPS with a custom domain, more detailed application monitoring, and better load-testing tools. I would also manage the EKS infrastructure with Terraform and implement a CI/CD pipeline to automate application deployments.

Architecture Explanation

Your application traffic follows this path:

User → Load Balancer → Kubernetes Service → Pods → Container

User: The user opens the application's public Load Balancer address in a browser. This sends an HTTP request over the internet to your EKS application.

Load Balancer: The AWS Load Balancer is the public entry point. It receives the request and forwards it into the EKS cluster through your `eks-project` Kubernetes Service.

Kubernetes Service: The Service provides a stable networking endpoint for the application. It receives traffic on port 80 and distributes requests across the available application Pods.

Pods: Pods are the running instances of your application. When HPA scales from 1 Pod to multiple Pods, the Service automatically distributes requests across those available Pods.

Container: Each Pod contains your Docker container running the Python application. The container listens on port 5000, processes the request, and sends the response back to the user.

Challenges Faced and Solutions

Helm deployment errors: Some required values and resources were incorrectly configured.

After troubleshooting, I updated `values.yaml`, disabled unnecessary HTTPRoute/Gateway API configuration, and successfully upgraded the Helm release.

HPA was not scaling initially: CPU usage was too low to reach the 70% target. **After troubleshooting**, I reduced the CPU request and generated traffic with load-generator Pods to trigger autoscaling.

Load-testing issues: Siege produced socket failures.

After troubleshooting, I used BusyBox with continuous HTTP requests for a more reliable load test.

CloudWatch setup issues: IAM/OIDC configuration and add-on installation caused permission errors.

After troubleshooting, I configured the EKS OIDC provider, IAM role/policies, and successfully installed the CloudWatch Observability add-on.

Old load generators affected testing: Previous load-generator Pods were still consuming resources.

After troubleshooting, I deleted them and restarted the test from a clean baseline.

CPU pressure without HPA: With autoscaling disabled, one Pod approached its CPU limit and response times increased.

After troubleshooting, I re-enabled HPA so Kubernetes could add Pods and distribute the workload.